

Template Tokens Special tokens that structure chat interactions (e.g., <start_of_turn> (abbreviated <sof>), user, model, <end_of_turn> (abbreviated <eot>)), delimiting user messages from model responses. Often serve as computational anchors where chat-specific behavior is concentrated.

Mathematical Notation

x	Input string or token sequence.
d	Dimension of model activations (residual stream dimension).
D	Number of latents in the crosscoder dictionary (typically $D \gg d$).
$\mathcal{J}$	Set of all latents $\{1, \dots, D\}$.
$\mathbf{h}^{\text{base}}(x)$	Base model activation vector at a specific layer for input x , where $\mathbf{h}^{\text{base}}(x) \in \mathbb{R}^d$.
$\mathbf{h}^{\text{chat}}(x)$	Chat model activation vector at the corresponding layer, where $\mathbf{h}^{\text{chat}}(x) \in \mathbb{R}^d$.
$f_j(x)$	Activation (scalar) of latent j for input x , where $f_j(x) \in \mathbb{R}_{\geq 0}$. Shared across both models in the crosscoder.
$\mathbf{d}_j^{\text{base}}$	Decoder direction for latent j in the base model, where $\mathbf{d}_j^{\text{base}} \in \mathbb{R}^d$. Represents how latent j contributes to base model activations.
$\mathbf{d}_j^{\text{chat}}$	Decoder direction for latent j in the chat model, where $\mathbf{d}_j^{\text{chat}} \in \mathbb{R}^d$. Can differ from $\mathbf{d}_j^{\text{base}}$ in both magnitude and direction.
$\tilde{\mathbf{h}}^{\text{base}}(x)$	Reconstructed base model activation: $\tilde{\mathbf{h}}^{\text{base}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{base}} + \mathbf{b}^{\text{dec,base}}$.
$\tilde{\mathbf{h}}^{\text{chat}}(x)$	Reconstructed chat model activation: $\tilde{\mathbf{h}}^{\text{chat}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{chat}} + \mathbf{b}^{\text{dec,chat}}$.
$\epsilon^{\text{base}}(x)$	Reconstruction error for base model: $\epsilon^{\text{base}}(x) = \mathbf{h}^{\text{base}}(x) - \tilde{\mathbf{h}}^{\text{base}}(x)$. Captures information not explained by the crosscoder.
$\epsilon^{\text{chat}}(x)$	Reconstruction error for chat model: $\epsilon^{\text{chat}}(x) = \mathbf{h}^{\text{chat}}(x) - \tilde{\mathbf{h}}^{\text{chat}}(x)$.
$\Delta_{\text{norm}}(j)$	Relative norm difference: $\Delta_{\text{norm}}(j) = \frac{1}{2} \left(1 + \frac{\ \mathbf{d}_j^{\text{chat}}\ _2 - \ \mathbf{d}_j^{\text{base}}\ _2}{\max(\ \mathbf{d}_j^{\text{chat}}\ _2, \ \mathbf{d}_j^{\text{base}}\ _2)} \right) \in [0, 1]$. Measures how chat-specific vs base-specific a latent is.
β_j^{base}	Optimal scaling factor for latent j to reconstruct base activations: minimizes $\sum_i \ \beta f_j(x_i) \mathbf{d}_j^{\text{chat}} - \mathbf{h}^{\text{base}}(x_i)\ _2^2$. Intuitively, how much the chat decoder helps explain base activations.
β_j^{chat}	Optimal scaling factor for latent j to reconstruct chat activations (analogous to β_j^{base})
ν_j	Overall scaling ratio: $\nu_j = \beta_j^{\text{base}} / \beta_j^{\text{chat}}$. Values near 0 indicate chat-specificity; values near 1 indicate equal presence in both models.
ν_j^r	Reconstruction ratio: $\nu_j^r = \beta_j^{r,\text{base}} / \beta_j^{r,\text{chat}}$, where β^r values are computed using reconstructions instead of raw activations. Detects Latent Decoupling (high values indicate the latent's information is captured by other base latents).
ν_j^ϵ	Error ratio: $\nu_j^\epsilon = \beta_j^{\epsilon,\text{base}} / \beta_j^{\epsilon,\text{chat}}$, where β^ϵ values are computed using errors. Detects Complete Shrinkage (high values indicate the latent should contribute to base reconstruction but doesn't).
p^{chat}	Chat model's next-token probability distribution given context
$p_{\mathbf{h}^{\text{chat}} \leftarrow \mathbf{h}_a}^{\text{chat}}$	Modified chat model distribution when activation $\mathbf{h}^{\text{chat}}$ is replaced with approximation $\tilde{\mathbf{h}}$

B Additional definitions

B.1 L1 crosscoder

L1 crosscoder. Let x be an string and $\mathbf{h}^{\text{base}}(x), \mathbf{h}^{\text{chat}}(x) \in \mathbb{R}^d$ denote the activations at a given layer at the last token of x . For a dictionary of size D , the latent activation of the j^{th} latent $f_j(x), j \in \mathcal{J} = \{1, \dots, D\}$ is computed as

$$f_j(x) = \text{ReLU}(\mathbf{e}_j^{\text{base}} \mathbf{h}^{\text{base}}(x) + \mathbf{e}_j^{\text{chat}} \mathbf{h}^{\text{chat}}(x) + b_j^{\text{enc}}) \quad (6)$$

where $\mathbf{e}_j^{\text{base}}, \mathbf{e}_j^{\text{chat}} \in \mathbb{R}^d$ are the corresponding encoder vectors and $b_j^{\text{enc}} \in \mathbb{R}$ is the encoder bias. The reconstructed activations for both models are then defined as:

$$\tilde{\mathbf{h}}^{\text{base}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{base}} + \mathbf{b}^{\text{dec,base}} \quad \text{and} \quad \tilde{\mathbf{h}}^{\text{chat}}(x) = \sum_j f_j(x) \mathbf{d}_j^{\text{chat}} + \mathbf{b}^{\text{dec,chat}} \quad (7)$$

where $\mathbf{d}_j^{\text{base}}, \mathbf{d}_j^{\text{chat}} \in \mathbb{R}^d$ are the j^{th} decoder latents and $\mathbf{b}^{\text{dec,base}}, \mathbf{b}^{\text{dec,chat}} \in \mathbb{R}^d$ are the decoder biases. We define the reconstruction errors for the base and chat models as $\epsilon^{\text{base}}(x) = \mathbf{h}^{\text{base}}(x) - \tilde{\mathbf{h}}^{\text{base}}(x)$ and $\epsilon^{\text{chat}}(x) = \mathbf{h}^{\text{chat}}(x) - \tilde{\mathbf{h}}^{\text{chat}}(x)$. The training loss for the L1 crosscoder is a modified L1 SAE objective, where μ controls the sparsity weight:

$$\mathcal{L}_{\text{L1}}(x) = \frac{1}{2} \|\epsilon^{\text{base}}(x_i)\|_2 + \frac{1}{2} \|\epsilon^{\text{chat}}(x_i)\|_2 + \mu \sum_j f_j(x) (\|\mathbf{d}_j^{\text{base}}\|_2 + \|\mathbf{d}_j^{\text{chat}}\|_2) \quad (8)$$

While similar to training an SAE on concatenated activations, the crosscoder’s sparsity loss uniquely promotes decoder norm differences (see Section C).

B.2 BatchTopK crosscoder

Let $\mathcal{X} = \{x_1, \dots, x_n\}$ be a batch of $|\mathcal{X}| = n$ inputs. Following Bussmann et al. [2024], we compute the latent activation function differently during training and inference. Let $f_j(x_i)$ be the latent activation function as defined in Equation (6). Given the scaled latent activation function $v(x_i, j) = f_j(x_i)(\|\mathbf{d}_j^{\text{base}}\|_2 + \|\mathbf{d}_j^{\text{chat}}\|_2)$, the training latent activation function f_j^{train} is given by:

$$f_j^{\text{train}}(x_i, \mathcal{X}) = \begin{cases} f_j(x_i) & \text{if } (x_i, j) \in \text{BATCHTOPK}(k, v, \mathcal{X}, \mathcal{J}) \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

where $\text{BATCHTOPK}(k, v, \mathcal{X}, \mathcal{J})$ represents the set of indices corresponding to the top $|\mathcal{X}| \cdot k$ values of the function v across all inputs $x_i \in \mathcal{X}$ and all latents $j \in \mathcal{J}$. We now redefine the reconstruction errors and the training loss for batch $\mathcal{X}$ as follows:

$$\epsilon^{\text{base}}(x_i, \mathcal{X}) = \mathbf{h}^{\text{base}}(x_i) - \left(\sum_j f_j^{\text{train}}(x_i, \mathcal{X}) \mathbf{d}_j^{\text{base}} + \mathbf{b}^{\text{dec,base}} \right) \quad (10)$$

$$\epsilon^{\text{chat}}(x_i, \mathcal{X}) = \mathbf{h}^{\text{chat}}(x_i) - \left(\sum_j f_j^{\text{train}}(x_i, \mathcal{X}) \mathbf{d}_j^{\text{chat}} + \mathbf{b}^{\text{dec,chat}} \right) \quad (11)$$

$$\mathcal{L}_{\text{BatchTopK}}(\mathcal{X}) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \|\epsilon^{\text{base}}(x_i, \mathcal{X})\|_2 + \frac{1}{2} \|\epsilon^{\text{chat}}(x_i, \mathcal{X})\|_2 + \alpha \mathcal{L}_{\text{aux}}(x_i, \mathcal{X}) \quad (12)$$

The auxiliary loss facilitates the recycling of inactive latents and is defined as $\|\epsilon^{\text{base}}(x_i, \mathcal{X}) - \hat{\epsilon}^{\text{base}}(x_i, \mathcal{X})\|_2 + \|\epsilon^{\text{chat}}(x_i, \mathcal{X}) - \hat{\epsilon}^{\text{chat}}(x_i, \mathcal{X})\|_2$, where $\hat{\epsilon}^{\text{base}}$ and $\hat{\epsilon}^{\text{chat}}$ represent reconstructions using only the top- k_{aux} dead latents. Typically, k_{aux} is set to 512 and α to 1/32. For inference, we employ the following latent activation function:

$$f_j^{\text{inference}}(x_i) = \begin{cases} f_j(x_i) & \text{if } v(x_i, j) > \theta \\ 0 & \text{otherwise} \end{cases} \quad (13)$$

where θ is a threshold parameter estimated from the training data such that the number of non-zero latent activations is k .

$$\theta = \mathbb{E}_{\mathcal{X}} \left[\min_{(x_i, j) \in \mathcal{X} \times \mathcal{J}} \{v(x_i, j) \mid f_j^{\text{train}}(x_i, \mathcal{X}) > 0\} \right] \quad (14)$$

B.3 Alternative BatchTopK variations

We experimented with several variations of the BatchTopK activation function to investigate whether alternative sparsity mechanisms could further improve the identification of *chat-specific* latents. However, none of these variations yielded more *chat-specific* latents than the BatchTopK approach described above, so we focus on this version in the main paper.

Concatenated decoder norm variant. The first variation modifies the scaling function $v(x_i, j)$ used in the top- k selection. Instead of summing the decoder norms as in our approach, we use the norm of the concatenated decoder vectors:

$$v'(x_i, j) = f_j(x_i) \|[\mathbf{d}_j^{\text{base}}, \mathbf{d}_j^{\text{chat}}]\|_2 \quad (15)$$

where $[\mathbf{d}_j^{\text{base}}, \mathbf{d}_j^{\text{chat}}] \in \mathbb{R}^{2d}$ denotes the concatenation of both decoder vectors. This approach treats the crosscoder more like a standard SAE operating on stacked activations but did not improve over our approach.

Model-independent BatchTopK variant. The second variation computes BatchTopK selection independently for each model, using the model-specific scaling function

$$v^M(x_i, j) = f_j(x_i) \|\mathbf{d}_j^M\|_2 \quad (16)$$

for model $M \in \{\text{base}, \text{chat}\}$. This approach was motivated by the observation that standard BatchTopK has an inherent bias toward shared latents. Since latents are selected based on their total reconstruction benefit across both models, a shared latent that reduces loss by 0.6 on each model (total benefit 1.2) will be preferred over a model-specific latent that reduces loss by 1.0 on one model and 0 on the other (total benefit 1.0). We hypothesized that this bias might prevent discovery of important chat-specific features introduced during fine-tuning, as they would be crowded out by shared representations. The model-independent variant removes this bias by allowing each model to allocate its k budget independently, potentially revealing chat-specific latents that would otherwise be suppressed. As expected, the model-independent variant produced more *chat-only* latents. However, these additional latents suffered from increased latent decoupling issues, ultimately not yielding more *chat-specific* latents by our ν^r and ν^ε metrics. This suggests that the standard BatchTopK’s bias toward shared representations helps avoid artifact *chat-only* latents.

C Comparing sparsity losses: Crosscoder vs. stacked SAE

An L1 crosscoder can be viewed as an SAE operating on stacked activations, where the encoder and decoder vectors are similarly stacked:

$$\mathbf{h}(x) = [\mathbf{h}^{\text{base}}(x), \mathbf{h}^{\text{chat}}(x)] \in \mathbb{R}^{2d} \quad (17)$$

$$\mathbf{e}_j = [\mathbf{e}_j^{\text{base}}, \mathbf{e}_j^{\text{chat}}] \in \mathbb{R}^{2d} \quad (18)$$

$$\mathbf{d}_j = [\mathbf{d}_j^{\text{base}}, \mathbf{d}_j^{\text{chat}}] \in \mathbb{R}^{2d} \quad (19)$$

$$\mathbf{b}^{\text{dec}} = [\mathbf{b}^{\text{dec,base}}, \mathbf{b}^{\text{dec,chat}}] \quad (20)$$

The reconstruction remains equivalent because

$$f_j(x) = \text{ReLU}(\mathbf{e}_j \mathbf{h} + b_j^{\text{enc}}) \quad (21)$$

$$\begin{aligned} &= \text{ReLU}(\mathbf{e}_j^{\text{base}} \mathbf{h}^{\text{base}}(x) + \\ &\quad \mathbf{e}_j^{\text{chat}} \mathbf{h}^{\text{chat}}(x) + b_j^{\text{enc}}) \end{aligned} \quad (22)$$

and hence,

$$[\tilde{\mathbf{h}}^{\text{base}}(x), \tilde{\mathbf{h}}^{\text{chat}}(x)] = \sum_j f_j(x) \mathbf{d}_j + \mathbf{b}^{\text{dec}} \quad (23)$$

However, the key difference arises in the sparsity loss. For the crosscoder, the sparsity loss is given by:

$$L_{\text{sparsity}}^{\text{crosscoder}}(x) = \sum_j f_j(x) \left(\sqrt{\sum_{i=1}^d (\mathbf{d}_{j,i}^{\text{chat}})^2} + \sqrt{\sum_{i=1}^d (\mathbf{d}_{j,i}^{\text{base}})^2} \right) \quad (24)$$

For a stacked SAE, it is:

$$\begin{aligned}
L_{\text{sparsity}}^{\text{SAE}}(x) &= \sum_j f_j(x) \sqrt{\sum_{i=1}^{2d} (\mathbf{d}_{j,i})^2} \\
&= \sum_j f_j(x) \sqrt{\sum_{i=1}^d (\mathbf{d}_{j,i}^{\text{base}})^2 + \sum_{i=1}^d (\mathbf{d}_{j,i}^{\text{chat}})^2}
\end{aligned} \tag{25}$$

The difference between $\sqrt{x+y}$ and $\sqrt{x} + \sqrt{y}$ introduces an inductive bias in the crosscoder that encourages the norm of one decoder (often the base decoder) to approach zero when the corresponding latent is only informative in one model.

Figure 9 displays a heatmap of the functions $\sqrt{x^2 + y^2}$ and $\sqrt{x^2} + \sqrt{y^2}$ along with their negative gradients, as visualized by the arrows. One can observe that for the crosscoder sparsity variant $\sqrt{x^2} + \sqrt{y^2}$ the gradient encourages the norm of one of the decoders to approach zero much more quickly compared to the SAE’s $\sqrt{x^2 + y^2}$.

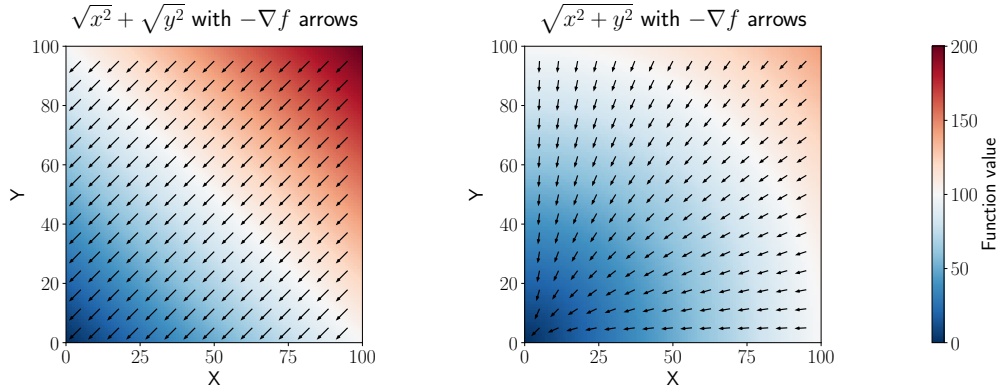


Figure 9: Heatmap comparing the two functions $\sqrt{x^2 + y^2}$ and $\sqrt{x^2} + \sqrt{y^2}$ along with their negative gradients.

D Illustrative example of Latent Decoupling

As a reminder, Latent Decoupling happens when a *chat-only* latent j is also present in the base activations but is reconstructed by other base decoder latents. To spell it out in more details, consider the following set up: a concept C may be represented identically in both models by some direction $\mathbf{d}_C$ but activate on different non-exclusive data subsets. Let $f_C^{\text{chat}}(x)$ and $f_C^{\text{base}}(x)$ be concept C ’s optimal activation functions in chat and base models, defined as $f_C^{\text{chat}}(x) = f_{\text{shared}}(x) + f_{\text{c-excl}}(x)$ and $f_C^{\text{base}}(x) = f_{\text{shared}}(x) + f_{\text{b-excl}}(x)$, where f_{shared} encodes shared activation, while $f_{\text{b-excl}}$ and $f_{\text{c-excl}}$ define model exclusive activations. For interpretability, the crosscoder should ideally learn three latents:

1. A *shared* latent j_{shared} representing C when active in both models using $f_{j_{\text{shared}}} = f_{\text{shared}}$ and $\mathbf{d}_{\text{chat}} = \mathbf{d}_{\text{base}} = \mathbf{d}_C$,
2. A *chat-only* latent j_{chat} representing C when exclusively active in the chat model using $f_{j_{\text{chat}}} = f_{\text{c-excl}}$ and $\mathbf{d}_{\text{chat}} = \mathbf{d}_C$, $\mathbf{d}_{\text{base}} = \mathbf{0}$, and
3. A *base-only* latent j_{base} representing C when exclusively active in the base model using $f_{j_{\text{base}}} = f_{\text{b-excl}}$ and $\mathbf{d}_{\text{chat}} = \mathbf{0}$, $\mathbf{d}_{\text{base}} = \mathbf{d}_C$.

However, the L1 crosscoder achieves equivalent loss using just two latents:

1. A *chat-only* latent j_{chat} representing C in the chat model using $f_{j_{\text{chat}}} = f_{\text{c-excl}} + f_{\text{shared}}$ and $\mathbf{d}_{\text{chat}} = \mathbf{d}_C$, $\mathbf{d}_{\text{base}} = \mathbf{0}$, and

2. A *base-only* latent j_{base} representing C in the base model using $f_{j_{\text{base}}} = f_{\text{b-excl}} + f_{\text{shared}}$ and $\mathbf{d}_{\text{chat}} = \mathbf{0}, \mathbf{d}_{\text{base}} = \mathbf{d}_C$. In this scenario, the so-called “*chat-only*” latent is only truly chat-only on a subset of its activation pattern.

Although whenever $f_{\text{shared}} > 0$ two latents are active instead of one, the sparsity loss is the same because the sparsity loss includes the decoder vector norms.¹² To illustrate the phenomenon of Latent Decoupling we choose the oversimplified case where $f_{\text{b-excl}}(x) = f_{\text{c-excl}}(x) = 0$. Let us consider a latent j with $f_j(x) = \alpha$. On the other hand, let there be two other latents p and q with

$$\begin{aligned} \mathbf{d}_p^{\text{base}} &= \mathbf{d}_j^{\text{base}}, & \mathbf{d}_p^{\text{chat}} &= \mathbf{0} \\ \mathbf{d}_q^{\text{base}} &= \mathbf{0}, & \mathbf{d}_q^{\text{chat}} &= \mathbf{d}_j^{\text{chat}} \end{aligned}$$

and $f_p(x) = f_q(x) = \alpha$. Clearly, the reconstruction is the same in both cases since $\alpha \mathbf{d}_j^{\text{base}} = \alpha \mathbf{d}_p^{\text{base}} + \alpha \mathbf{d}_q^{\text{base}}$ and $\alpha \mathbf{d}_j^{\text{chat}} = \alpha \mathbf{d}_q^{\text{chat}} + \alpha \mathbf{d}_q^{\text{chat}}$. Further, the L1 regularization term is the same since

$$\alpha (\|\mathbf{d}_j^{\text{base}}\|_2 + \|\mathbf{d}_j^{\text{chat}}\|_2) = \tag{26}$$

$$\begin{aligned} &\alpha (\|\mathbf{d}_p^{\text{base}}\|_2 + \|\mathbf{d}_p^{\text{chat}}\|_2) \\ &+ \alpha (\|\mathbf{d}_q^{\text{base}}\|_2 + \|\mathbf{d}_q^{\text{chat}}\|_2) \\ &= \alpha (\|\mathbf{d}_p^{\text{base}}\|_2 + 0) + \alpha (0 + \|\mathbf{d}_q^{\text{chat}}\|_2) \end{aligned} \tag{27}$$

Hence both solutions achieve the exact same loss under the L1 crosscoder.

However, the BatchTopK crosscoder actively encourages the three-latent solution. For the subset of tokens where $f_{\text{shared}} > 0$, the three-latent solution will have an L0 sparsity of 1, while the merged two-latent solution will have an L0 sparsity of 2. Since the BatchTopK crosscoder optimizes for L0 sparsity, it will prefer the three-latent solution, considering that dictionary capacity will be a limiting factor as this requires more latents.

E More details regarding Latent Scaling

E.1 Closed form solution for Latent Scaling

Consider a latent j with decoder vector $\mathbf{d}$. Our goal is to find the optimal scaling factor β that minimizes the squared reconstruction error:

$$\underset{\beta}{\operatorname{argmin}} \sum_{i=0}^n \|\beta f(x_i) \mathbf{d} - \mathbf{y}\|_2^2 \tag{28}$$

To solve this optimization problem efficiently, we reformulate it in matrix form. Let $\mathbf{Y} \in \mathbb{R}^{n \times d}$ be the stacked data matrix and $\mathbf{f} \in \mathbb{R}^n$ be the vector of latent activations for latent j across all datapoints. The objective can then be expressed using the Frobenius norm of the residual matrix $\mathbf{R} = \beta \mathbf{f} \mathbf{d}^T - \mathbf{Y}$, where $\mathbf{f} \mathbf{d}^T \in \mathbb{R}^{n \times d}$ represents the outer product of the latent activation vector and decoder vector. Our minimization problem becomes:

$$\begin{aligned} \|\mathbf{R}\|_F^2 &= \|\beta \mathbf{f} \mathbf{d}^T - \mathbf{Y}\|_F^2 \\ &= \operatorname{Tr} [(\beta \mathbf{f} \mathbf{d}^T - \mathbf{Y})^\top (\beta \mathbf{f} \mathbf{d}^T - \mathbf{Y})] \\ &= \operatorname{Tr} [\mathbf{Y}^\top \mathbf{Y}] - 2\beta \operatorname{Tr} [\mathbf{Y}^\top \mathbf{f} \mathbf{d}^T] \\ &\quad + \beta^2 \operatorname{Tr} [(\mathbf{f} \mathbf{d}^T)^\top \mathbf{f} \mathbf{d}^T] \end{aligned}$$

Using trace properties, we get:

¹²In the simplest case where $f_{\text{c-excl}}(x) = f_{\text{b-excl}}(x) = 0$, there exists a *base-only* latent j_{twin} with $\mathbf{d}_j^{\text{chat}} = \mathbf{d}_{j_{\text{twin}}}^{\text{base}}$ and identical activation function that reconstructs the information of $\mathbf{d}_j^{\text{chat}}$ in the base model. The sparsity loss equals that of a single shared latent.